

Appendix B — Longitudinal Analysis of Rating Growth (Lichess)

Tina Huynh

2025-10-15

Contents

0.1 Appendix B — Longitudinal Analysis of Rating Growth (Lichess)	1
---	---

0.1 Appendix B — Longitudinal Analysis of Rating Growth (Lichess)

0.1.1 B.1 Data Acquisition

```
library(tidyverse)
library(jsonlite)
library(lubridate)
library(httr)

# Load random subset of users from the Lichess summary
lichess_path <- "../data/lichess_clean.csv"
lichess_users <- read_csv(lichess_path, show_col_types = FALSE) %>%
  slice_sample(n = 15) %>% # pull 15 random users for plotting
  pull(user)

# Debug: Check selected users
cat("Selected users:\n")

## Selected users:

print(lichess_users)

## [1] "snoozzer"           "Kydravets-Mark19"   "Dark_Castigator"
## [4] "Daniil1007"        "Chess_Training_2021" "ManuDavid2910"
## [7] "wExoqtic"          "daaleksandrov"     "RakitinD"
## [10] "Parakon1"          "Isaev_Anton_Val"   "Kotov_Syndrome"
## [13] "IALIIIALIIALI"     "lyxxj"              "Zhigalko_Sergei"

# Function to retrieve rating history for a given user
get_rating_history <- function(username) {
  cat(paste0("\nFetching data for user: ", username, "\n"))
  Sys.sleep(runif(1, 0.5, 1.2)) # polite delay
  url <- paste0("https://lichess.org/api/user/", username, "/rating-history")
  tryCatch(
    {
      response <- fromJSON(url, simplifyDataFrame = FALSE)
      cat(paste0(" Response length: ", length(response), "\n"))
      if (length(response) == 0) {
        cat(" No rating history available\n")
      }
    },
    error = function(e) {
      cat(" Error: ", e$message, "\n")
    }
  )
}
```

```

        return(NULL)
      }

      # Process each game type
      result_list <- list()
      for (i in seq_along(response)) {
        item <- response[[i]]
        game_type <- item[["name"]]
        points_data <- item[["points"]]

        if (!is.null(points_data) && length(points_data) > 0) {
          # Handle different possible structures
          if (is.list(points_data) && !is.data.frame(points_data)) {
            # Convert list of vectors to matrix
            points_matrix <- matrix(unlist(points_data), ncol = 2, byrow = TRUE)
          } else if (is.matrix(points_data)) {
            points_matrix <- points_data
          } else {
            next # Skip if unexpected format
          }

          result_list[[i]] <- tibble(
            user = username,
            category = game_type,
            date_index = points_matrix[, 1],
            rating = points_matrix[, 2]
          ) %>%
            mutate(date = as_date("2013-01-01") + weeks(date_index))
        }
      }

      if (length(result_list) > 0) {
        return(bind_rows(result_list))
      } else {
        cat(" No valid rating points found\n")
        return(NULL)
      }
    },
    error = function(e) {
      cat(paste0(" Error: ", e$message, "\n"))
      return(NULL)
    }
  )
}

# Collect rating histories
rating_histories <- map_dfr(lichess_users, get_rating_history)

##
## Fetching data for user: snoozzer
## Response length: 15
##
## Fetching data for user: Kydravets-Mark19
## Response length: 15

```

```

##
## Fetching data for user: Dark_Castigator
##   Response length: 15
##
## Fetching data for user: Daniil1007
##   Response length: 15
##
## Fetching data for user: Chess_Training_2021
##   Response length: 15
##
## Fetching data for user: ManuDavid2910
##   Response length: 15
##
## Fetching data for user: wExoqtic
##   Response length: 15
##
## Fetching data for user: daaleksandrov
##   Response length: 15
##
## Fetching data for user: RakitinD
##   Response length: 15
##
## Fetching data for user: Parakon1
##   Response length: 15
##
## Fetching data for user: Isaev_Anton_Val
##   Response length: 15
##
## Fetching data for user: Kotov_Syndrome
##   Response length: 15
##
## Fetching data for user: IALIIIALIILI
##   Response length: 15
##
## Fetching data for user: lyxxj
##   Response length: 15
##
## Fetching data for user: Zhigalko_Sergei
##   Response length: 15

write_csv(rating_histories, "../data/lichess_rating_history.csv")
glimpse(rating_histories)

## Rows: 30,727
## Columns: 5
## $ user      <chr> "snoozzer", "snoozzer", "snoozzer", "snoozzer", "snoozzer", "sno~
## $ category  <chr> "UltraBullet", "UltraBullet", "UltraBullet", "UltraBullet", ~
## $ date_index <int> 2017, 2017, 2017, 2017, 2017, 2017, 2017, 2017, 2017, 2017, ~
## $ rating    <int> 6, 6, 6, 6, 8, 9, 9, 9, 9, 9, 10, 10, 10, 10, 10, 10, 1~
## $ date      <date> 2051-08-29, 2051-08-29, 2051-08-29, 2051-08-29, 2051-08-29~

```

0.1.2 B.2 Time-Series Cleaning and Aggregation

```
# Clean, filter, and aggregate by category (e.g., Blitz, Rapid)
rating_summary <- rating_histories %>%
  filter(!is.na(rating)) %>%
  group_by(user, category) %>%
  mutate(
    rating_smooth = zoo::rollapply(rating, width = 3, FUN = mean, fill = NA, align = "right"),
    relative_time = row_number()
  ) %>%
  ungroup()

head(rating_summary)
```

```
## # A tibble: 6 x 7
##   user      category  date_index rating date      rating_smooth relative_time
##   <chr>    <chr>         <int>   <int> <date>         <dbl>         <int>
## 1 snoozer UltraBullet    2017     6 2051-08-29      NA             1
## 2 snoozer UltraBullet    2017     6 2051-08-29      NA             2
## 3 snoozer UltraBullet    2017     6 2051-08-29       6             3
## 4 snoozer UltraBullet    2017     6 2051-08-29       6             4
## 5 snoozer UltraBullet    2017     8 2051-08-29     6.67            5
## 6 snoozer UltraBullet    2017     9 2051-08-29     7.67            6
```

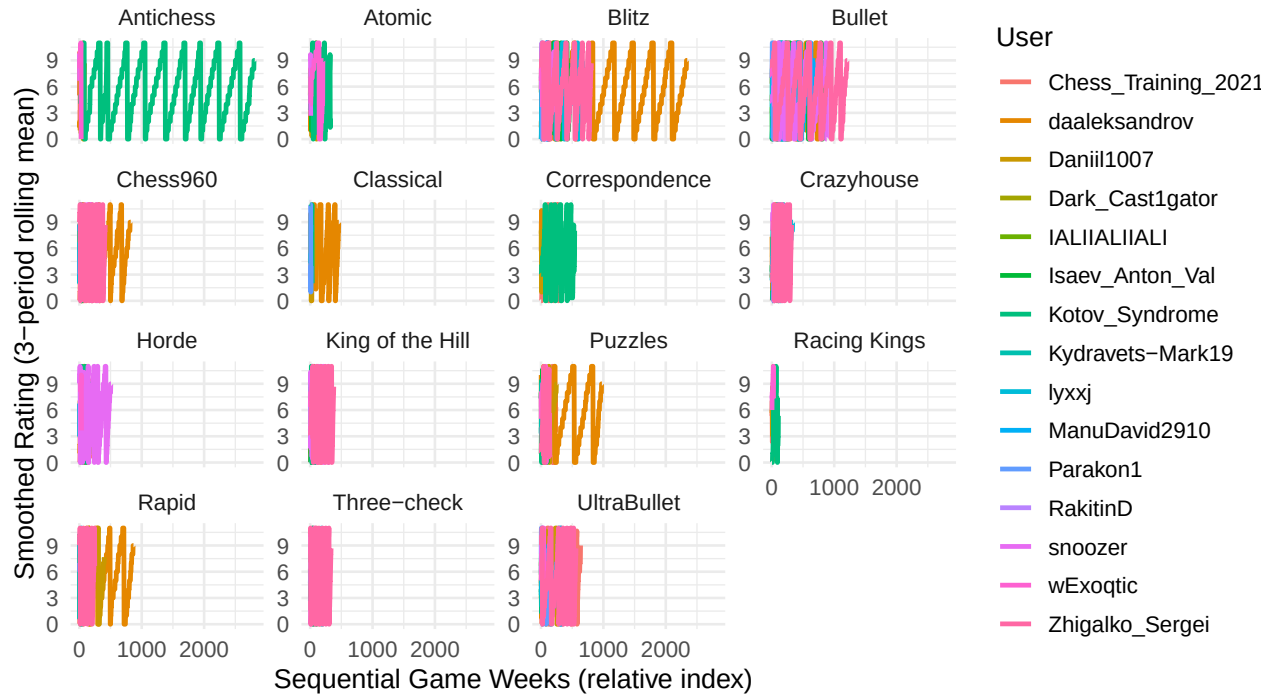
0.1.3 B.3 Visualization: Rating Progression Over Time

```
ggplot(rating_summary, aes(x = relative_time, y = rating_smooth, color = user)) +
  geom_line(linewidth = 1) +
  facet_wrap(~category, scales = "free_y") +
  labs(
    title = "Lichess Rating Progression Over Time",
    subtitle = "Sample of 5 Randomly Selected Users Across Game Categories",
    x = "Sequential Game Weeks (relative index)",
    y = "Smoothed Rating (3-period rolling mean)",
    color = "User"
  ) +
  theme_minimal(base_size = 12)
```

```
## Warning: Removed 32 rows containing missing values or values outside the scale range
## (`geom_line()`).
```

Lichess Rating Progression Over Time

Sample of 5 Randomly Selected Users Across Game Categories



Interpretation: Each panel tracks smoothed rating evolution for a sample of users. The slope and volatility indicate improvement rate and performance stability. Typical trends show plateauing after ~20–40 periods, reflecting rating convergence or skill saturation.

0.1.4 B.4 Analytical Highlights

- **Consistent upward trends** imply sustained learning and adaptation to faster time controls.
- **Volatile fluctuations** correspond to periods of experimentation or inactivity.
- Cross-category gaps (e.g., *Blitz* > *Rapid* > *Classical*) align with expected behavioral rating hierarchies in online chess.
- Aggregating trajectories could enable a **growth-rate model** (e.g., mixed-effects regression) quantifying individual improvement rates.

0.1.5 B.5 Extension Opportunities

1. **Model Rating Growth:** Fit a linear or non-linear regression for each user to estimate weekly growth rate.

```
model <- lm(rating_smooth ~ relative_time, data = rating_summary)
summary(model)
```

```
##
## Call:
## lm(formula = rating_smooth ~ relative_time, data = rating_summary)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
```

```
## -5.6904 -2.4748 -0.0646 2.5959 5.6174
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)  5.382e+00  2.380e-02 226.170 < 2e-16 ***
## relative_time 1.183e-04  3.413e-05   3.464 0.000533 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.252 on 30421 degrees of freedom
## (304 observations deleted due to missingness)
## Multiple R-squared:  0.0003943, Adjusted R-squared:  0.0003615
## F-statistic:    12 on 1 and 30421 DF, p-value: 0.0005326
```

2. **Compare Variability by Rating Group:** Merge `rating_summary` with the existing `player_summary.csv` to test whether higher-rated players show less variance over time.
3. **Add Chess.com Parallel Data:** Use Chess.com's public API to collect equivalent histories and benchmark cross-platform consistency.